



Object-Oriented Enterprise Application Development

— ■ — ■ — ■ — ■ — ■ — ■ — ■ — ■ — ■ — ■ —
Introduction to JavaBeans
— ■ — ■ — ■ — ■ — ■ — ■ —

Topics

✧ JavaBeans

- ◆ Introduction
 - What are they?
 - Why should you care?
- ◆ Merging with servlets
 - Benefits
- ◆ Merging with JSPs
 - Benefits
 - Techniques
- ◆ This will not be comprehensive!

What are JavaBeans?

- ✧ A Java **Bean** is a reusable software component that can be manipulated in a builder tool.
- ✧ **JavaBeans** is Java's component model.
 - ◆ Components are self-contained units that can be assembled to form complex systems.
 - ◆ These components must interoperate according to a specified set of rules and guidelines.
- ✧ JavaBeans provides the core concepts behind Enterprise JavaBeans (EJBs).

JavaBean Concepts

- ✧ Each component is called a **bean**.
- ✧ A bean's capabilities can be discovered at run time without the developer knowing anything about the bean ahead of time.
- ✧ Like other objects, beans use their interface to allow their state to be changed during run time.
- ✧ Bean generally provide some mechanism for saving and retrieving their internal state.

JavaBean Concepts (cont.)

✧ To work effectively with beans, we need some basic concepts.

- ◆ **Properties.** The data manipulated by the bean. The bean's internal state.
- ◆ **Methods.** The methods representing the bean's interface.
- ◆ **Events.** The mechanism by which one bean sends notifications to another. This is commonly done through the user of **callbacks**.

JavaBean Concepts (cont.)

- ✧ Beans are generally used at run time. Therefore there must be a mechanism by which a client of the bean can learn about that bean.
- ✧ That process is called **introspection**.
 - ◆ Introspection is the process of exposing a bean's properties, methods, and events.
- ✧ Instead of introspection, we can also provide an explicit **BeanInfo** class (discussed later).

JavaBean Concepts (cont.)

- ✧ Beans are often asked to save their state to some non-volatile medium such as a disk or database.
- ✧ This is known as **persistence**.
- ✧ Each JavaBean must provide some sort of persistence mechanism.
 - ◆ One approach is to use Java Object Serialization by implementing the **java.io.Serializable** interface.
 - ◆ This is not the only approach.

JavaBean Concepts (cont.)

✧ JavaBeans are classes that encapsulate many objects into a single object (the bean). It is a Java class that should follow the following conventions:

- ◆ Must implement Serializable.
- ◆ It should have a public no-arg constructor.
- ◆ All properties in java bean must be private with public getters and setter methods.

JavaBean Example

```
public class Student implements java.io.Serializable {  
  
    private int id;    private String name;  
  
    // Constructor  
    public Student() {}  
  
    // Setter for Id  
    public void setId(int id) { this.id = id; }  
  
    // Getter for Id  
    public int getId() { return id; }  
  
    // Setter for Name  
    public void setName(String name) { this.name = name; }  
  
    // Getter for Name  
    public String getName() { return name; }  
}
```

JavaBean Example

```
public class Test {  
    // main function  
    public static void main(String args[])  
    {  
        // object is created  
        Student s = new Student();  
  
        // setting value to the object  
        s.setName("My Student");  
  
        System.out.println(s.getName());  
    }  
}
```

Output: **My Student**

JavaBean with JSP

- ✧ The bean implemented on the following page is a very simple JavaBean using JSP.
- ✧ It contains a single property, three methods, and no events.
 - ◆ The single property is both readable and writable.
- ✧ It uses Java Object Serialization.

PhraseBean.java

```
public class PhraseBean
implements java.io.Serializable {
    private String phrase;

    public PhraseBean() {
        phrase = "Hello, World!";
    }

    public void setPhrase(String phrase) {
        this.phrase = phrase;
    }

    public String getPhrase() {
        return (phrase);
    }
}
```


PhraseBean

✧ This is a legitimate JavaBean.

✧ JavaBeans can implement the **java.io.Serializable** interface if they require persistence.

- ◆ This allows the bean to be persisted if necessary.
- ◆ This isn't the only persistence mechanism available.

✧ All beans should have a default constructor.

- ◆ This allows us to create them using the **instantiate()** method (discussed later).

Using the PhraseBean

- ✧ A logical place to use this kind of bean is a JSP.
 - ◆ Since a bean is just a Java class, it can be used anywhere that a Java class can be used.
 - ◆ Accessing a bean within a JSP simply requires a bit more work.
- ✧ To use the bean we must first create it and then trigger one of its accessor methods.
- ✧ Once a bean has been created we can use its properties and methods.

Phrase.jsp

```
-----  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0  
    Transitional//EN">  
<HTML>  
    <HEAD><TITLE>Phrase JSP</TITLE></HEAD>  
    <BODY>  
        <jsp:useBean id="phrase" class="se452.PhraseBean" />  
        <H2>  
            <P>Old phrase is  
                <jsp:getProperty name="phrase"  
                    property="phrase" />  
            </P>  
            <jsp:setProperty name="phrase"  
                property="phrase"  
                value = "Hello again, World!" />  
            <P>New phrase is  
                <jsp:getProperty name="phrase"  
                    property="phrase" />  
            </P>  
        </H2>  
    </BODY></HTML>
```

useBean Tag

- ✦ The **useBean** tag creates a new instance of the bean specified by the **class** attribute and associates it with the name provided by the **id** attribute.
- ✦ The example in the code creates a new instance of the **se452.PhraseBean** class and assigns it to a variable called **phrase**:

```
<jsp:useBean id="phrase" class="se452.PhraseBean"/>
```

Or

```
<jsp:useBean id="phrase" class="se452.PhraseBean"  
  scope="Session"/>
```


useBean Tag (cont)

✧ Scope

- ◆ Page
- ◆ Request
- ◆ Session
- ◆ Application

✧ Bean automatically initialize and referenced in the declared scope

- ◆ The same id in the same scope is not allowed

getProperty Tag

- ✦ The **getProperty** tag retrieves the property specified by the **property** attribute from the bean specified by the **name** attribute.
- ✦ Before you can get a bean's properties, that bean must be created with the **useBean** tag.
- ✦ The example in the code retrieves the current value of the **phrase** bean's **phrase** property:

```
<jsp:getProperty name="phrase" property="phrase"/>
```


getProperty Tag

✦ Instead of using the **getProperty** tag:

```
<jsp:getProperty name="phrase" property="phrase"/>
```

we could have used a JSP expression:

```
<P>Old phrase is  
    <%= phrase.getPhrase() %>  
</P>
```

✦ Using the **getProperty** tag makes it clear that we are using a bean and not another generic object.

setProperty Tag

- ✦ The **setProperty** tag allows us to change the property specified by **property** for the bean specified by **name** to the value specified by **value**.
- ✦ Before you can set a bean's properties, that bean must be created with the **useBean** tag.
- ✦ The example in the code sets the current value of the **phrase** bean's **phrase** property:

```
<jsp:setProperty name="phrase"  
                 property="phrase"  
                 value = "Hello again, World!"/>
```


setProperty Tag

✦ Instead of using the **setProperty** tag:

```
<jsp:setProperty name="phrase"  
                 property="phrase"  
                 value="Hello again, World!" />
```

we could have used a JSP scriptlet:

```
<% phrase.setPhrase("Hello again, World!") %>
```

✦ Using the **setProperty** tag makes it clear that we are using a bean and not another generic object.

Properties and Introspection

- ✧ You may have noticed that the **getProperty** and **setProperty** tags are referencing the bean's properties, which are nothing more than private data to the bean's class.
- ✧ Behind the scenes however, the JSP is converting **property** attributes to method calls:
 - ◆ **setProperty**: changes **phrase** property to **setPhrase** method call.
 - ◆ **getProperty**: changes **phrase** property to **getPhrase** method call.

Linking Forms and Beans

✧ Since beans are really where the program logic for a JSP should reside, it makes sense that we should have some mechanism for linking bean properties to user input entered on an HTML form.

✧ One method might be the following:

```
<jsp:setProperty name="bean" property="beanprop"  
  value = "<% request.getParameter("formparam") %>"  
/>
```

✧ This approach is cumbersome at best although it is also extremely flexible.

Linking Forms and Beans

- ✧ Because we want to link forms and beans so often, there are several shortcuts for doing so.
- ✧ If the form element has the same name as a bean property we can use the following syntax:

```
<jsp:setProperty name="bean" property="formparam" />
```
- ✧ The JSP will look for a property on the bean called **formparam** and will assume that the value comes from a request parameter with the same name.

Linking Forms and Beans

- ✧ Often, the element names on a form don't match the properties on a bean.
- ✧ In this case we can use the following variation of the **setProperty** tag:

```
<jsp:setProperty name="bean" property="beanprop"  
    param="formparam" />
```

- ✧ The JSP will look for a request parameter called **formparam** and will assign it to the bean's **beanprop** property.

Linking Forms and Beans

✦ The last method of linking forms to bean properties asks the JSP engine to search through all the request parameters and to link them to the corresponding bean properties.

✦ In this case we can use the following variation of the **setProperty** tag:

```
<jsp:setProperty name="bean" property="*" />
```

✦ This leaves too much to chance, particularly if the elements on the form, for whatever reason, don't match to the properties on the bean.

Serialization

- ✧ To be considered as a bean, a class must provide a means of serializing itself.
- ✧ This allows the bean's state to be **serialized**, or persisted. even though the JVM in which the bean executes shuts down.
- ✧ The most common way of performing this serialization is to have the bean implement the **java.io.Serializable** interface.
 - ◆ There are other ways of providing this persistence.

Serialization (cont.)

- ✧ The bean saves itself to a special file called a **serial file**. All of its properties will be saved to this file.
 - ◆ Each property to be saved must also be serializable.
- ✧ If a given property should not be serialized, then it needs to be marked as transient.
 - ◆ Properties that maintain state-specific data such as socket or database connections shouldn't be serialized.
 - ◆ We can't guarantee that when they're re-hydrated, that their state will make sense.

Using a Serialized Bean

- ✧ Using a serialized bean is similar to using a standard bean. There is a slight change to the **useBean** tag:

```
<jsp:useBean id="phrase" beanName="phrase"  
type="se452.PhraseBean"/>
```

- ✧ The **beanName** attribute identifies the file in which the serialized bean is stored.
 - ◆ By default an extension of **.ser** is used for persisted beans although you never explicitly state this.

Configuring Tomcat

- ✧ At present Jakarta-Tomcat doesn't support the use of serialized beans.
- ✧ This is an active work-in-process with their group.

Bean Scope

- ✦ By default beans only have scope local to the page on which they are created.
- ✦ This means that, by default, it isn't possible for a bean on one page to change a value in a bean on another page and have that value appear on that page.
- ✦ There are ways to change this default behavior by changing the bean's **scope**.

Page Scope

- ✦ The default scope for beans is **page** scope. A bean with page scope can be created by using a slightly modified **useBean** tag that includes the **scope** attribute:

```
<jsp:useBean id="phrase" class="se452.PhraseBean"  
scope="page" />
```

- ✦ Each time the page is requested, the bean will be created, be manipulated by the JSP code, and disappear when the JSP is done executing.

Request Scope

- ✦ We can create our beans so that they last as long as the request object for the JSP exists:

```
<jsp:useBean id="phrase" class="se452.PhraseBean"
scope="request"/>
```

- ✦ This means that the bean will be available even if the request is sent to another JSP using the **forward()** or **include()** methods on the **RequestDispatcher** object.

Session Scope

- ✧ It is common to want to store information so that whenever a user visits some location of the site, their information is available.
- ✧ In servlets we used the **HttpSession** object to capture this data.
- ✧ To make beans available this way, we use the **session** scope:

```
<jsp:useBean id="phrase" class="se452.PhraseBean" scope="session"/>
```


Session Scope

- ✧ When dealing with **session** scope, there are sometimes cases where we only want a bean to perform some action when it is first created.
 - ◆ For instance we might want to capture some initialization data based on the request object such as the user's browser configuration.
- ✧ For **page** and **request** scopes we always get a new bean so this behavior makes no sense, but for **session** scope we need to control this behavior.

useBean Tag Revisited

- ✧ We can configure our JSP to only perform certain bean initialization code when the bean is first constructed.
- ✧ This involves the use of an end tag for **useBean** start tag.

Phrase.jsp

```
-----  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">  
<HTML>
```

```
  <HEAD><TITLE>Phrase JSP</TITLE></HEAD>
```

```
  <BODY>
```

```
    <jsp:useBean id="phrase" class="se452.PhraseBean"  
      scope="session">
```

```
      <jsp:setProperty name="phrase" property="phrase"  
        value="Hello first time, World!" />
```

```
    </jsp:useBean>
```

```
    <H2>
```

```
      <P>Old phrase is
```

```
        <jsp:getProperty name="phrase" property="phrase" />  
      </P>
```

```
      <jsp:setProperty name="phrase" property="phrase"  
        value = "Hello again, World!" />
```

```
      <P>New phrase is
```

```
        <jsp:getProperty name="phrase" property="phrase" />  
      </P>
```

```
    </H2>
```

```
  </BODY></HTML>
```

useBean Tag Revisited

- ✧ Any JSP code, including HTML statements, declarations, scriptlets, and bean accessors or modifiers, will only be executed when the specified bean is first created.
- ✧ Since this is based on the user's session, each user will get this first-time behavior no matter how many user's have visited the page before them.

Application Scope

- ✦ Sometimes we want data to be visible to all users and across multiple pages.
- ✦ Maintaining multiple beans that all store the same data isn't efficient in terms of resources.
- ✦ To make beans available across the entire application, irrespective of user or page, we use the **application** scope:

```
<jsp:useBean id="phrase" class="se452.PhraseBean"  
scope="application"/>
```

Typecasting

- ✧ All of the data sent to the JSP via the request object is String data.
- ✧ If a bean property expects data other than String data, the JSP engine will attempt to perform an explicit typecast using pre-defined conversion rules.
- ✧ These typecasts could fail.
 - ◆ Strings containing letters cannot be successfully converted to integers.

Typecasting

- ✦ If a conversion fails, then the mutator method will not be called but an exception will not necessarily be raised.
- ✦ Obviously this is not desirable; our bean could be in an inconsistent state which could easily cause runtime problems later on.
- ✦ One solution is to use indicator variables within the bean to determine if a property was set successfully.

CountBean.java

```
import java.io.*;
```

```
public class CountBean  
implements java.io.Serializable {  
    private int count;  
    private boolean countOk;  
  
    public CountBean() {  
        count = 0;  
        countOk = true;  
    }  
}
```


CountBean.java

```
public void setCount(String count) {  
    try {  
        this.count = Integer.parseInt(count);  
        this.countOk = true;  
    }  
    catch (Exception exc) {  
        this.countOk = false;  
    }  
}  
  
public int getCount() {  
    return (count);  
}
```

CountBean.java

```
public boolean isValid() {  
    return (countOk);  
}  
  
public String getReason() {  
    if (!countOk) {  
        return ("Last.attempt.to.set.count.failed.");  
    }  
    else {  
        return ("");  
    }  
}  
}
```


Count.jsp

```
-----  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0  
    Transitional//EN">  
<HTML>  
    <HEAD><TITLE>Count JSP</TITLE></HEAD>  
    <BODY>  
        <jsp:useBean id="countBean" class="se452.CountBean"  
            scope="session" />  
        <jsp:setProperty name="countBean" property="count"  
            param= "countfield" />  
        <%  
            String text = null;  
            if (!countBean.isValid())  
                text = countBean.getReason();  
            else  
                text = "new count is " + countBean.getCount();  
        %>  
        <H2><%= text %></H2>  
    </BODY></HTML>
```

Events

- ✦ Originally beans were designed to be graphical components such as buttons, slider bars, and lists.
- ✦ This lead to the development of an event model where these classes could listen for events in which they were interested.
- ✦ This is the same idea as that behind most GUI development involving AWT and Swing.
- ✦ It turns out that the event model has relevance to server side processing as well.

Event Concepts

✦ There are three key elements to the event model:

- ◆ **Event objects.** This encapsulates the information that's specific to an instance of a given event. This includes such information as the event's source.
- ◆ **Event sources.** The object that triggered, or fired, the event in the first place.
- ◆ **Event listeners.** An object that is notified when a specific type of event has been fired.

Interaction

✦ The event model works as show in this diagram:



Events in Java

✧ The **java.util** package contains the basic classes used by the event model.

- ◆ **java.util.EventObject**

- ◆ **java.util.EventListener**

✧ There is no specific class for an event source. Instead, when an object wishes to trigger an event, it invokes a callback method on each listener that is interested in that event.

EventObject Class

- ✧ All events extend **`java.util.EventObject`**.
 - ◆ This class contains important information such as:
 - A reference to the event source.
 - ◆ It also provides essential capabilities such as:
 - Retrieving the event source.
 - Providing itself as a human-readable string.
- ✧ By convention, all of your event subclasses should end in the word **Event**.

PhraseChangedEvent Class

- ✧ Suppose that I want to be able to notify listeners that the PhraseBean's phrase has been changed.
- ✧ To enable this, I first create a new event class called **PhraseChangedEvent**.
- ✧ This class need not provide any new functionality, so all its going to do is provide a single-argument constructor.

PhraseChangedEvent.java

```
import java.util.*;

public class PhraseChangedEvent
extends java.util.EventObject
{
    PhraseChangedEvent(Object eventSource)
    {
        super(eventSource);
    }
}
```


Listeners

-
- ✧ A listener is simply an object that is notified whenever one of a set of events occurs in which that listener is interested.
 - ✧ This is the basic principle behind all web, application, and database servers.
 - ◆ When a request comes in on the appropriate port, a web server knows that it has work to do.

PhraseChangeListener Interface

- ✧ The new **PhraseChangeEvent** object is fairly simple and doesn't provide much functionality.
- ✧ Nevertheless, it is sufficient to allow a **PhraseBean** object to notify any listeners that it might have that the phrase has changed.
- ✧ The next step is to define an interface to be implemented by objects that listen for these notifications.
 - ◆ This will be the **PhraseChangeListener** interface.

PhraseChangeListener.java

```
import java.util.*;

public interface PhraseChangeListener
extends java.util.EventListener
{
    public void phraseChanged(PhraseChangedEvent
event);
}
```

EventListener Class

- ✧ The **`java.util.EventListener`** interface contains no methods. It is simply used to indicate that a given interface is used to listen for events.
- ✧ It is a good idea to extend it into your own interface instead of implementing it directly.
 - ◆ This allows for a hierarchy of interfaces to be defined. Each interface provides only those method signatures in which it is interested.
 - ◆ Listeners can then implement only the interfaces they need.

PhraseListener Class

- ✧ Now that we have a listener interface, we can implement a concrete listener.
- ✧ This will be the **PhraseListener** class.
 - ◆ Its job is to register itself with a phrase bean and be notified when a phrase changes.

PhraseListener.java

```
import java.util.*;
```

```
public class PhraseListener
implements PhraseChangeListener
{
    public void
    phraseChanged(PhraseChangedEvent event)
    {
        System.out.println("Phrase.bean.has.changed");
    }
}
```


Event Sources

- ✦ There are no special classes to extend or interfaces to implement for an event source.
- ✦ It does need to provide the ability for event listeners to register and de-register themselves.
- ✦ This is done by providing two key methods:

```
public void add<ListenerType>(<ListenerType listener);
```

- This registers the listener with the event source.

```
public void remove<ListenerType>(<ListenerType listener);
```

- This de-registers the listener with the event source.

PhraseBean Revisited

- ✦ In order to allow the PhraseBean class to act as an event source, we need to implement the following two methods:

```
public void addPhraseChangeListener(  
    PhraseChangeListener listener);  
  
public void removePhraseChangeListener(  
    PhraseChangeListener listener);
```

- ✦ Note that the method names are by convention only. However, following this convention will ensure compatibility with all third-party tools used to assemble beans.

PhraseBean.java

```
import java.io.*;
import java.util.*;

public class PhraseBean
implements java.io.Serializable {
    private String phrase = null;
    private Vector phraseChangeListeners = null;

    public PhraseBean() {
        phrase = "Hello, World!";
        phraseChangeListeners = new Vector();
    }

    public void setPhrase(String phrase) {
        this.phrase = phrase;
        notifyPhraseChange();
    }
}
```

PhraseBean.java (cont.)

```
public String getPhrase() {  
    return (phrase);  
}
```

```
public synchronized void  
addPhraseListener(PhraseListener listener) {  
    if (!phraseChangeListeners.contains(listener)) {  
        phraseChangeListeners.addElement(listener);  
    }  
}
```

```
public synchronized void  
removePhraseListener(PhraseListener listener) {  
    if (phraseChangeListeners.contains(listener)) {  
        phraseChangeListeners.removeElement(listener);  
    }  
}
```


PhraseBean.java (cont.)

```
protected void notifyPhraseChange() {  
    PhraseChangedEvent event = new  
        PhraseChangedEvent(this);  
  
    Vector vec;  
    synchronized(this) {  
        vec = (Vector)phraseChangeListeners.clone();  
    }  
  
    int count = vec.size();  
    for (int i = 0; i < count; i++) {  
        PhraseListener client =  
            (PhraseListener)vec.elementAt(i);  
        client.phraseChanged(event);  
    }  
}
```

PhraseListener Revisited

- ✧ Now that the **PhraseBean** can support the notification of each of its listeners, we can allow the **PhraseListener** class to register itself with the **PhraseBean**.
- ✧ The basic change is that when a listener is created it is given a reference to a bean. This is what allows the listener to be notified when the bean changes.

PhraseListener.java

```
import java.util.*;
```

```
public class PhraseListener
implements PhraseChangeListener {
    private PhraseBean phraseBean;

    public PhraseListener(PhraseBean phraseBean) {
        this.phraseBean = phraseBean;
        phraseBean.addPhraseListener(this);
    }

    public void phraseChanged(PhraseChangedEvent
event) {
        System.out.println("Phrase.bean.has.changed");
    }
}
```

Testing the Events

- ✦ Testing the events consists of creating a bean and then using it as the event source for one or more event listeners.
- ✦ The next example show a single bean acting as the event source for multiple listeners.

TestPhrase.java

```
public class TestPhrase {  
    public static void main(String [] args) {  
        PhraseBean bean = new PhraseBean();  
        PhraseListener listener =  
            new PhraseListener(bean);  
        PhraseListener listener2 =  
            new PhraseListener(bean);  
  
        bean.setPhrase("this.is.a.new.phrase");  
    }  
}
```

Java Bean in MVC architecture

✧ MVC is the most popular pattern architecture for WEB application.

✧ MVC – Model – View – Controller

- ◆ Model: responsibility for processing business data
 - Java Bean – DAO: Business data, Tier Data
- ◆ View: responsibility for data presentation and direct user interaction
 - JSP – JSTL – EL – Custom tags
- ◆ Controller: responsibility for controlling and connecting component
 - Servlet – Filter - Listener

Review

✧ During this class we have:

- ◆ Seen how to construct and use basic JavaBeans.
- ◆ Seen how to integrate JavaBeans components into JSPs.
- ◆ Seen how to write events that allow one bean to trigger actions on another using the JavaBeans event model.